实验1：编写矩阵乘法

实验目的

1. 理解计算机指令集架构、地址模式；
2. 通过实现矩阵乘法程序学习并应用 x86-64 汇编语言中的不同寻址方式；
3. 理解在硬件层面如何访问和操作内存。

实验前准备

复习汇编语言的基本概念，重点了解如寄存器间接寻址、基址+变址寻址、立即数寻址和比例变址寻址等寻址方式。

实验环境：安装了汇编器和调试器的计算机。

实验内容

1. 编写简易汇编程序，熟悉和验证汇编实验环境；
2. 使用提供的汇编代码实现矩阵乘法，并调整代码以处理不同大小的矩阵；
3. 通过操作系统命令、文件和评估工具获取系统目标处理器的缓存信息；
4. 学习使用perf观察程序性能。

实验原理

1. 矩阵乘法的遍历方式

假设矩阵 A 大小为 $M \times K$，矩阵 B 大小为 $K \times N$。根据《线性代数》的定义，矩阵 A 和矩阵 B 的相乘的结果为 $M \times N$ 大小的矩阵 C，矩阵 C 的元素可由下述公式计算：

$$C_{ij} = \sum_{0 \le k < K} A_{ik} \cdot B_{kj} \qquad 0 \le i < M,\ 0 \le j < N$$

假设矩阵 A、矩阵 B 和矩阵 C 可按《线性代数》中分块矩阵乘法的定义划分成如图2-1所示的子矩阵。由分块矩阵乘法的定义可知 $C_{00} = A_{00} \cdot B_{00} + A_{01} \cdot B_{10} + A_{02} \cdot B_{20}$。要完成 C_{00} 的计算必须沿 K 维度完成矩阵 A 和矩阵 B 所有分块的点乘的计算。

一般情况下，通常会先遍历结果矩阵的两个维度，最后再遍历剩下的一个维度。如上面介绍的矩阵 C 元素的计算公式就是典型的 ijk 遍历方式。这种遍历方式的优点在于最后一个维度遍历完即可完成矩阵 C 的一个元素的计算。

除了 ijk 遍历方式外，kij 遍历方式（也称秩 K 更新）的使用也较为普遍。不同于 ijk 遍历方式，使用 kij 遍历方式计算矩阵 C 时需要存中间结果。由图2-1可知，遍历矩阵 A 的一列和矩阵 B 的一行，只能得到矩阵 C 每个元素的一部分结果，只有当矩阵 A 和矩阵 B 的所有分块全遍历完时才能得到矩阵 C 的最终结果。相较于 ijk 遍历方式，kij 遍历方式能较好的保证空间访问的局部性，提高缓存的利用率。

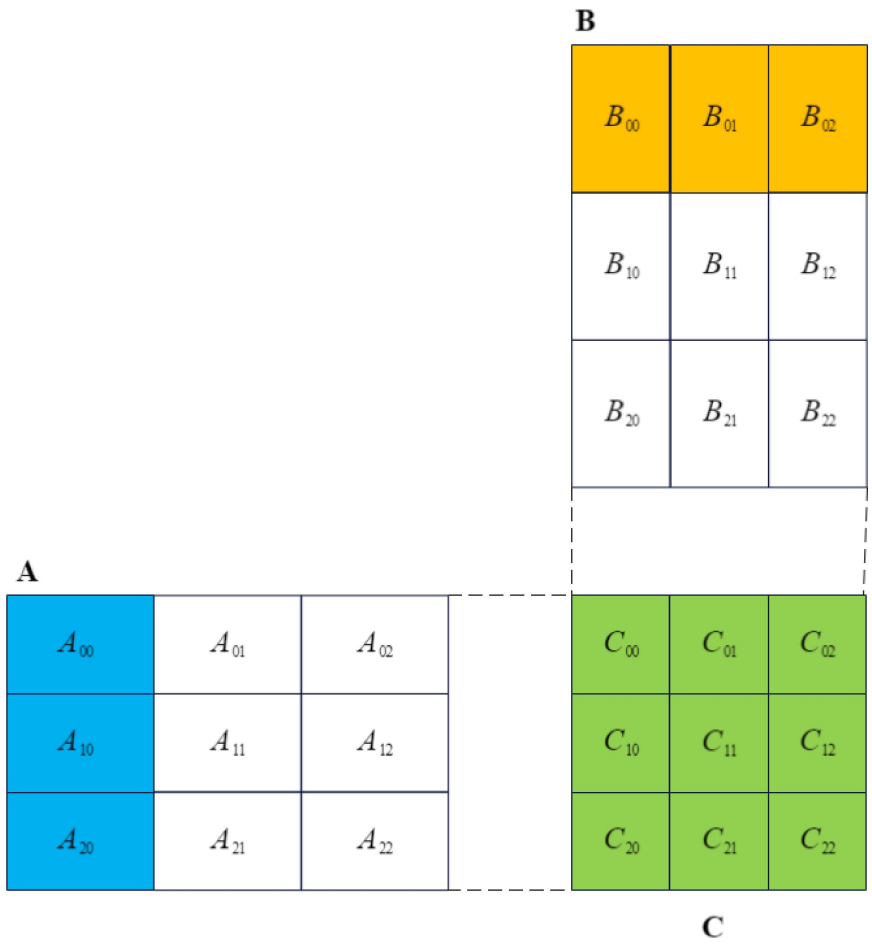


图2-1 kij 遍历的矩阵乘法算法

2. x86-64简述

x86-64是x86的64位扩展，其在保持良好兼容性的基础上将寄存器和虚拟地址空间扩展到64位，并提升了指令集的功能和性能。

2.1 寄存器概述

x86-64架构包含状态和控制寄存器、程序指针寄存器、通用寄存器、向量寄存器等多种类型的寄存器，如表2-1所示。

表2-1 x86-64架构的部分寄存器

类型	名称	位宽	说明

控制和状态寄存器	RFLAGS	64	处理器标志位信息，包括零标志（ZF）、符号位（SF）、进位位（CF）、溢出位（OF）、中断使能位（IF）等
指针寄存器	RIP	64	当前指令的地址
通用寄存器	RAX	64	常用作累加、存放**乘积或被除数的低64位**、存放**除法的商**，或传递**子程序的返回值**
	RBX	64	常用作基数寄存器
	RCX	64	常用作计数器，或传递**子程序调用时的第4个参数**
	RDX	64	常用作数据寄存器、存放**乘积或被除数的高64位**、存放**除法的余数**，或传递**子程序调用时的第3个参数**
	RSI	64	常用作字符串操作的源索引（Source Index）寄存器，或传递**子程序调用时的第2个参数**
	RDI	64	常用作字符串操作的目标索引（Destination Index）寄存器，或传递**子程序调用时的第1个参数**
	RSP	64	栈指针
	RBP	64	栈基址指针
	R8~R15	64	通用；R8、R9分别用作传递**子程序调用时的第5、6个参数**
向量寄存器	XMM0~XMM15	128	SSE (Streaming SIMD Extension) 寄存器；YMM的低128位
	YMM0~YMM15	256	AVX (Advanced Vector eXtension) 寄存器

2.2 寻址方式

x86-64架构的常用寻址方式包括立即数寻址、寄存器寻址、直接寻址、寄存器间接寻址、基址变址寻址和比例变址寻址，如表2-2所示。

表2-2 x86-64架构的寻址方式

寻址方式	使用方法	说明
立即数寻址	`mov $0x1234, %rax`	RAX ← `0x1234`
寄存器寻址	`mov %r8, %rax`	RAX ← R8
直接寻址	`mov 0x1234, %rax`	RAX ← MEM[`0x1234`]
寄存器间接寻址	`mov (%rsp), %rax`	RAX ← MEM[RSP]
基址变址寻址	`mov -20(%rsp), %rax`	RAX ← MEM[RSP - 20]
比例变址寻址	`mov -20(%rsp, %rcx, 4), %rax`	RAX ← MEM[RSP - 20 + RCX*4]

由表2-2可知，直接寻址、寄存器间接寻址和基址变址寻址均可视为比例变址寻址的特殊情况。

2.3 常用指令



1

x86-64 Reference Sheet

David Broman, KTH Royal Institute of Technology
Version 0.2, December 1, 2020

Operands

Forms	Examples	
1. Registers:	%rax, %edx, %di	
2. Immediate operands:		
Decimal form:	$-3	
Hexadecimal form:	$0x21af	21afh
Binary form:	$0b11010111	1101_0111b
3. Memory operands		
(num)	(0x1232)	
(reg)	(%rbx)	
(base, index, scale)	(%rbx, %rcx, 4)	
num(base)	9(%rbx)	
num(base, index, scale)	2(%rbx, %rcx, 8)	scale ∈ {1, 2, 4, 8}

Only 64-bit registers are allowed in memory operations

Integer Instructions (subset, AT&T)

Inst	Operands	Description	Semantics
add	op2, op1	Addition	op1RM := op1RM + op2RMI
and	op2, op1	Bitwise AND	op1RM := op1RM & op2RMI
call	op1	Procedure call	rsp := rsp - 8; [rsp] := PC; PC := op1RMI
cdq		Sign extend 32-bit reg.	edx:eax := signext(eax)
cmovC	op2, op1	Conditional move	op1R := op1RM if condition C
cmp	op2, op1	Compare	op1RM - op2RMI
cqo		Sign extend 64-bit reg.	rdx:rax := signext(rax)
dec	op1	Decrement by 1	op1RM := op1RM - 1
div	op1	Unsigned division	rax (rdx) := rdx:rax / op1RM
idiv	op1	Signed division	rax (rdx):= rdx:rax / op1RM
imul	op1	Signed multiplication	rdx:rax := rax * op1RM
inc	op1	Increment by 1	op1RM := op1RM + 1
jC	op1	Conditional jump	PC := op1I if C else next
jmp	op1	Unconditional jump	PC := op1RMI
mov	op2, op1	Move	op1RM := op2RMI
movsx	op2, op1	Move w. sign-extension	op1R := signext(op2RM)
movzx	op2, op1	Move w. zero-extension	op1R := zeroext(op2RM)
mul	op1	Unsigned multiplication	rdx:rax := rax * op1RM
neg	op1	Signed negation	op1RM := 0 - op1RM
not	op1	Bitwise NOT operation	op1RM := ~op1RM
or	op2, op1	Bitwise OR (inclusive)	op1RM := op1RM | op2RMI
pop	op1	Pop from stack	op1RM := [rsp]; rsp := rsp + 8
push	op1	Push on stack	rsp := rsp - 8; [rsp] := op1RMI
ret		Return from procedure	PC := [rsp]; rsp := rsp + 8
sar	op2, op1	Shift arithmetic right	op1RM := op1RM >>> op2RI
setC	op1	Set byte on condition	op1R := 1 if condition C is true, else 0
shl	op2, op1	Shift logical left	op1RM := op1RM << op2RI
shr	op2, op1	Shift logical right	op1RM := op1RM >> op2RI

Registers

#	64-bit	32-bit	16-bit	8-bit	Note	Function
0	rax	eax	ax	al	-	Accumulator
3	rbx	ebx	bx	bl	Callee saved	Base
1	rcx	ecx	cx	cl	Param 4	Counter
2	rdx	edx	dx	dl	Param 3	Data
6	rsi	esi	si	sil	Param 2	Source Index
7	rdi	edi	di	dil	Param 1	Destination Index
5	rbp	ebp	bp	bpl	Callee saved	Base Pointer
4	rsp	esp	sp	spl	-	Stack Pointer
8	r8	r8d	r8w	r8b	Param 5	-
9	r9	r9d	r9w	r9b	Param 6	-
10	r10	r10d	r10w	r10b	-	-
11	r11	r11d	r11w	r11b	-	-
12	r12	r12d	r12w	r12b	Callee saved	-
13	r13	r13d	r13w	r13b	Callee saved	-
14	r14	r14d	r14w	r14b	Callee saved	-
15	r15	r15d	r15w	r15b	Callee saved	-

- The # column can be used for numbered registers, e.g. r3 = rbx. Note that in NASM, line `%use altreg` is needed for numbered registers.
- Reg eax represents the least significant bits (LSB) of rax, ax the LSB of eax, and al is the LSB of ax. Same pattern holds for all other registers.

Calling Conventions

- Integer arguments: rdi, rsi, rdx, rcx, r8, r9
- Floating-point arguments: xmm0 - xmm7
- Additional arguments pushed on stack, right to left, removed by caller
- Callee saved registers: rbp, rbx, r12, r13, r14, r15
- Integer return register(s): rax or rdx:rax
- Floating-point number return register(s): xmm0 or xmm1:xmm0

Directives and Comments

.text	Assembly instructions (code section)
.data	Initialized data (data section)
.bss	Uninitialized data (bss section)
.global label	Make the label visible to the outside
.extern label	Assumes that the label is defined elsewhere
; comment	Line comments start with a semicolon

Data Reservation

Reservations in the .data section

db	0x1f	Initializes 1 byte
db	1, 78, 0x2f	Initializes 3 bytes
db	'A', 10	Initializes 2 bytes
db	"Hello", 0	Initializes 6 bytes

sub	op2, op1	Subtraction	op1RM := op1RM - op2RMI
xchg	op2, op1	Exchange (swap)	t := op1RM; op1RM = op2RM; op2RM := t (t is temp)
xor	op2, op1	Bitwise XOR (exclusive)	op1RM := op1RM ^ op2RMI

Notes

- Meaning of op1RMI: op1 = the operand, R = can be a register, M = can be a memory operand, and I = can be an immediate op. Example: op1RM allows reg and mem, but not immediate value.

dw	0x1821	Initializes a 16-bit word (2 bytes)
dd	0x1782ab12	Initializes a 32-bit word (4 bytes)
dq	0x8, 0x3	Initializes two 64-bit words (16 bytes)

Reservations in the .bss section

resb	88	Reservers 88 uninitialized bytes
resw	10	Reservers 10 uninitialized 16-bit words (20 bytes)
resd	10	Reservers 10 uninitialized 32-bit words (40 bytes)
resq	10	Reservers 10 uninitialized 64-bit words (80 bytes)

3. x87 FPU

3.1 寄存器概述

x87 FPU浮点运算单元包含8个80位的通用寄存器 `R0` ~ `R7`、3个16位的状态和控制寄存器、2个48位的指针寄存器以及1个11位的操作码寄存器，如图2-2所示。

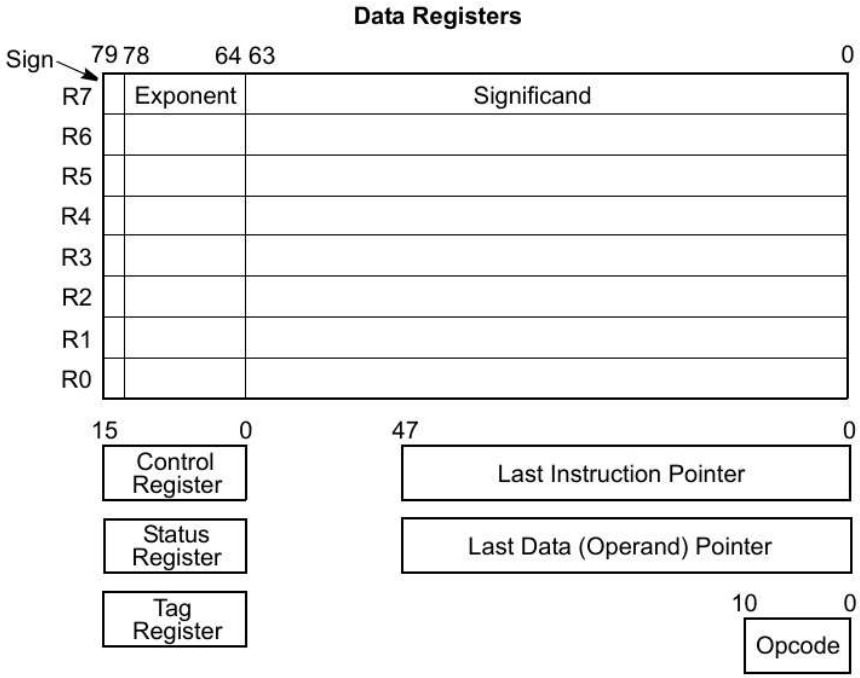


图2-2 x87 FPU的寄存器

其中，通用寄存器用于存储计算数据。数据被写入通用寄存器时将被FPU自动扩展至80位的双扩展精度浮点数（Double Extended-Precision Floating-Point）；而当通用寄存器中的数据被写入内存时，可以在不同精度的浮点数、整数或BCD码的多种格式当中灵活选择一种进行写入。

x87 FPU指令支持以栈的形式访问8个通用寄存器，且处于栈顶的数据始终被标记为 `st(0)`，如图2-3中的 (a) ~ (d)；同时也支持通过相对 `st(0)` 的偏移量来访问任意一个通用寄存器，如图2-3中的 (e)。

计算$5.6 \times 2.4 + 3.8 \times 10.3$

```
val1 = 5.6
val2 = 2.4
val3 = 3.8
val4 = 10.3
```

```
1| FLD  val1   ; (a) 将val1压入栈中
2| FMUL val2   ; (b) st(0) *= val2
3| FLD  val3   ; (c) 将val3压入栈中
4| FMUL val4   ; (d) st(0) *= val3
5| FADD ST(1)  ; (e) st(0) += st(1)
```

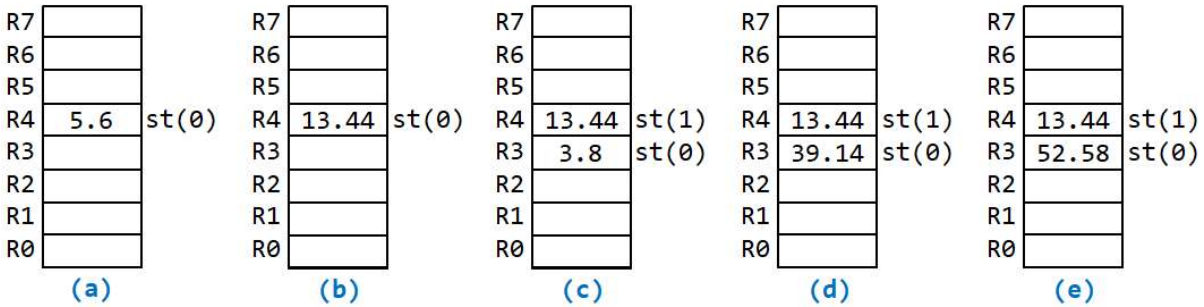


图2-3 通用寄存器访问示意图

3.2 常用指令

x87 FPU支持加载指令、存储指令和运算指令。加载指令将数据压入通用寄存器栈。存储指令将栈中数据弹出并存储到主存单元。运算指令实现浮点数据的加减乘除和求相反数等操作。

x87 FPU指令的命令规则是前缀 `f` + 前缀 `i` (可选) + 操作 + 后缀 `p` / `s` / `l` / `t` (可选) 。其中，前缀 `i` 、后缀 `s` / `l` / `t` 分别表示指令的操作数是整数、单精度浮点数、双精度浮点数和双扩展精度浮点数；后缀 `p` 表示执行该指令时需要在运算完成后从通用寄存器栈中弹出一个数据。

x87指令示例

- 例1： `fistpl (%rax)` 表示将 `st(0)` 中的双精度浮点数转换成整数后存储到 `%rax` 寄存器指向的存储单元，并从FPU栈中弹出一个数据。
- 例2： `faddp %st(3), %st(0)` 表示将 `st(3) + st(0)` 的计算结果保存在 `st(3)` ，然后把 `st(0)` 弹栈。



1

Index

CS:APP2e , 1

architecture
floating-point, *1*

`fabs` [IA32/x86-64] FP absolute value, 8
`fadd` [IA32/x86-64] FP add, 8
`fchs` [IA32/x86-64] FP negate, 8
`fcom` [IA32/x86-64] FP compare, 12
`fcoml` [IA32/x86-64] FP compare double precision, 12
`fcomp` [IA32/x86-64] FP compare with pop, 12
`fcompl` [IA32/x86-64] FP compare double precision with pop, 12
`fcompp` [IA32/x86-64] FP compare with two pops, 12
`fcomps` [IA32/x86-64] FP compare single precision with pop, 12
`fcoms` [IA32/x86-64] FP compare single precision, 12
`fcos` [IA32/x86-64] FP cosine, 8
`fdiv` [IA32/x86-64] FP divide, 8
`fdivr` [IA32/x86-64] FP reverse divide, 8
`fildl` [IA32/x86-64] FP load and convert integer, 6
`fistl` [IA32/x86-64] FP convert and store integer, 7
`fistpl` [IA32/x86-64] FP convert and store integer with pop, 7
`fisubl` [IA32/x86-64] FP load and convert integer and subtract, 8
`fld1` [IA32/x86-64] FP load one, 8
`fldl` [IA32/x86-64] FP load double precision, 6
`fldl` [IA32/x86-64] FP load from register, 6
`flds` [IA32/x86-64] FP load single precision, 6
`fnstw` [IA32/x86-64] copy FP status word, *13*
`fsin` [IA32/x86-64] FP sine, 8
`fsqrt` [IA32/x86-64] FP square root, 8
`fst` [IA32/x86-64] FP store to register, 7
`fstl` [IA32/x86-64] FP store double precision, 7
`fstp` [IA32/x86-64] FP store to register with pop, 7
`fstpl` [IA32/x86-64] FP store double precision with pop, 7
`fstps` [IA32/x86-64] FP store single precision with pop, 7
`fstpt` [IA32/x86-64] FP store extended precision with pop, 7
`fsts` [IA32/x86-64] FP store single precision, 7
`fstt` [IA32/x86-64] FP store extended precision, 7
`fsub` [IA32/x86-64] FP subtract, 8
`fsubl` [IA32/x86-64] FP load double precision and subtract, 8
`fsubp` [IA32/x86-64] FP subtract with pop, 8
`fsubr` [IA32/x86-64] FP reverse subtract, 8
`fsubs` [IA32/x86-64] FP load single precision and subtract, 8
`fsubt` [IA32/x86-64] FP load extended precision and subtract, 8
`fucom` [IA32/x86-64] FP unordered compare, 12
`fucoml` [IA32/x86-64] FP unordered compare double precision, 12
`fucomp` [IA32/x86-64] FP unordered compare with pop, 12
`fucompl` [IA32/x86-64] FP unordered compare double precision with pop, 12
`fucompp` [IA32/x86-64] FP unordered compare with two pops, 12
`fucomps` [IA32/x86-64] FP unordered compare single precision with pop, 12

fldt [IA32/x86-64] FP load extended precision, 6
fldz [IA32/x86-64] FP load zero, 8
floating point
registers in x87, *2*
status word, 12
floating-point architecture, *1*
fmul [IA32/x86-64] FP multiply, 8
fucomis [IA32/x86-64] FP unordered compare single precision, 12
fxch [IA32/x86-64] FP exchange registers, *7*

IEEE 754 floating-point standard, 2

registers
x87 floating point, *2*

练习1：环境搭建与初步测试

- **任务**：编写一个简单的程序打印一个整数，以验证汇编环境。
- **目标**：
  - 确保正确安装和配置工具库；
  - 熟悉汇编语言的语法和编译过程。

> ⚠ **实验环境说明** 💻
>
> 本课程实验需要在实验室的 **Ubuntu双系统** 上完成，不要使用虚拟机、WSL等。
>
> 开机时，请选择Ubuntu系统，而不是默认的Windows系统。

0. 下载并解压实验包

从首页链接下载 `lab1.tar.gz`，将其拷贝至用户目录，并执行命令 `tar -zxvf lab1.tar.gz` 解压实验包。

1. 安装编译所需的基础环境

首先执行下列命令更新软件源：

```
sudo apt update
```

执行下列命令安装环境：

```
sudo apt install -y build-essential net-tools git vim cmake gdb make gfortran libnuma-dev libtirpc-dev
```

2. 测试环境安装是否成功

执行下列命令并查看是否能输出每个工具的版本信息。

```
cmake -version
gcc --version
```

```
gdb --version
```

3. 为 `src/lab1/print_integer.S` 源码中的 `convert_loop` 循环(源码第33行)添加正确的结束指令

```
    mov %dl, (%rdi)             // 结果写内存
                                // 判断是否结束，TODO: 请添加判断结束的指令
    jnz convert_loop
```

4. 修复 `src/lab1/print_integer.S` 源码中的 `print_string` 过程中(第48行-第55行)存在错误

```
    // 系统调用号 (sys_write)     // TODO: 请修复此处的系统调用以正常输出字符串
    mov $1, %rax
    // 文件描述符 (stdout)
    mov $1, %rdi
    // 指向字符串的指针
    mov %rdi, %rsi
    // 要写入的字节数
    mov %rdx, %rdx
    // 执行系统调用
    syscall
```

5. 进入项目根目录，编译代码

```
cd lab1
mkdir -p build && cd build
cmake -B . -S ../ && cmake --build ./ --target lab1_print_integer
```

以上第3条命令需从校内网克隆git仓库，因此请确保计算机能正常访问校园网。

6. 运行代码得到正确输出

```
./dist/bins/lab1_print_integer
```

```
root@f07f7979017b:/data/computer_architecture_labs/build/dist/bins# ./lab1_print_integer
12345678901234567890
```

练习2：实现矩阵乘法

- **任务**：使用提供的汇编代码实现矩阵乘法，并调整代码以处理不同大小的矩阵。
- **目标**：
  - 理解寄存器间接寻址和基址+变址寻址方式；
  - 初始化矩阵数据，并在汇编中使用指针。

1. 为 `src/lab1/gemm_kernel.S` 中的 `GEMM_INIT` 过程（第56行）添加合适的矩阵B地址保存指令

```
55    .macro GEMM_INIT                                    // 初始化
56        // TODO: 将矩阵B的地址存入MAT_B宏对应的寄存器
```

2. 为 `src/lab1/gemm_kernel.S` 中的 `DO_GEMM` 过程（第70行）添加加载 `A[m][k]` 到FPU寄存器栈的逻辑

```
67    DO_LOOP_M:                                          // M维度的循环
68        xor loop_n, loop_n                              // 清空M维度的循环计数器
69
70        // TODO:  加载A[m][k]到st(0)
```

3. 为 `src/lab1/gemm_kernel.S` 中的 `DO_GEMM` 过程（第77行）添加加载 `B[k][n]` 到FPU寄存器栈的逻辑

```
76    DO_LOOP_N:
77        // TODO:  加载B[k][n]
78
79
80
81        flds (MAT_B, mat_elem_idx, 4)                   // 加载 B[k][n]
```

4. 为 `src/lab1/gemm_kernel.S` 中的 `DO_GEMM` 过程（第85行）添加加载 `C[m][n]` 到FPU寄存器栈的逻辑

```
85      // TODO: 加载C[m][n]
86
87
88
89      flds (MAT_C, mat_elem_idx, 4)                   // 加载 C[m][n]
```

5. 验证矩阵乘法内核的正确性

编译和测试之前，请先在终端通过 `cd` 命令进入 `lab1/build` 目录。

执行命令编译测试程序：

```
cmake -B . -S ../ && cmake --build ./ --target lab1_test_gemm_kernel.unittest
```

运行测试：

```
./dist/bins/lab1_test_gemm_kernel.unittest --gtest_filter=gemm_kernel.test0
```

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build/dist/bins$ ./lab1_test_gemm_kernel.unittest
--gtest_filter=gemm_kernel.test0
Running main() from /home/lenovo/test1/TA/try/computer_archtecture_labs/build/_deps/googletest-src/goo
gletest/src/gtest_main.cc
Note: Google Test filter = gemm_kernel.test0
[==========] Running 1 test from 1 test suite.
[----------] Global test environment set-up.
[----------] 1 test from gemm_kernel
[ RUN      ] gemm_kernel.test0
[       OK ] gemm_kernel.test0 (73 ms)
[----------] 1 test from gemm_kernel (73 ms total)

[----------] Global test environment tear-down
[==========] 1 test from 1 test suite ran. (73 ms total)
[  PASSED  ] 1 test.
```

6. 编译上层代码形成二进制程序

编译和测试之前，请先在终端通过 `cd` 命令进入 `lab1/build` 目录。

执行命令编译测试程序：

```
cmake -B . -S ../ && cmake --build ./ --target lab1_gemm
```

7. 运行代码得到输出

```
./dist/bins/lab1_gemm 256 256 256
```

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build/dist/bins$ ./lab1_gemm 256 256 256
GEMM performance info:
                M, K, N: 256, 256, 256
                Ops: 0.0335544
                Total compute time(s): 3.40523
                Cost(s): 0.0170262
                Benchmark(Gflops): 1.97076
```

练习3：获取目标处理器的缓存信息

- **任务**：通过操作系统提供的命令、文件和评估工具获取系统目标处理器的缓存信息。
- **子任务**：
  - 获取目标处理器缓存层级和各级缓存大小信息；
  - 获取目标处理器各级缓存的组相联数、缓存行大小信息。
- **目标**：学会获取未知架构的处理器的缓存信息。

1. 通过 `lscpu` 命令查看处理器型号、缓存层级信息

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build/dist/bins$ lscpu
架构:                           x86_64
CPU 运行模式:                   32-bit, 64-bit
字节序:                         Little Endian
Address sizes:                  39 bits physical, 48 bits virtual
CPU:                            12
在线 CPU 列表:                  0-11
每个核的线程数:                 2
每个座的核数:                   6
座:                             1
NUMA 节点:                      1
厂商 ID:                        GenuineIntel
CPU 系列:                       6
型号:                           158
型号名称:                       Intel(R) Core(TM) i7-8700 CPU @ 3.20GHz
步进:                           10
CPU MHz:                        800.007
CPU 最大 MHz:                   4600.0000
CPU 最小 MHz:                   800.0000
BogoMIPS:                       6399.96
虚拟化:                         VT-x
L1d 缓存:                       192 KiB
L1i 缓存:                       192 KiB
L2 缓存:                        1.5 MiB
L3 缓存:                        12 MiB
NUMA 节点0 CPU:                 0-11
Vulnerability Itlb multihit:    KVM: Vulnerable
Vulnerability L1tf:             Mitigation; PTE Inversion; VMX vulnerable
Vulnerability Mds:              Vulnerable; SMT vulnerable
Vulnerability Meltdown:         Vulnerable
Vulnerability Spec store bypass: Vulnerable
Vulnerability Spectre v1:       Vulnerable: __user pointer sanitization and usercopy barriers only; n
                                o swapgs barriers
Vulnerability Spectre v2:       Vulnerable, IBPB: disabled, STIBP: disabled
Vulnerability Tsx async abort:  Vulnerable
标记:                           fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse
                                36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb
                                 rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopo
```

```
logy nonstop_tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl
 vmx smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4_2 x2a
pic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm
 abm 3dnowprefetch cpuid_fault epb invpcid_single ssbd ibrs ibpb stib
p tpr_shadow vnmi flexpriority ept vpid ept_ad fsgsbase tsc_adjust bm
i1 hle avx2 smep bmi2 erms invpcid rtm mpx rdseed adx smap clflushopt
 intel_pt xsaveopt xsavec xgetbv1 xsaves dtherm ida arat pln pts hwp
hwp_notify hwp_act_window hwp_epp md_clear flush_l1d
```

2. 查看cpu0的L1D缓存的组数、组相联数和缓存行大小

```
cd /sys/devices/system/cpu/cpu0/cache
cd index0
# 查看缓存行大小
cat coherency_line_size
# 查看组数
cat number_of_sets
# 查看way数
cat ways_of_associativity
```

```
lenovo@10:/sys/devices/system/cpu/cpu0/cache$ cd index0
lenovo@10:/sys/devices/system/cpu/cpu0/cache/index0$ cat coherency_line_size
64
lenovo@10:/sys/devices/system/cpu/cpu0/cache/index0$ cat number_of_sets
64
lenovo@10:/sys/devices/system/cpu/cpu0/cache/index0$ cat ways_of_associativity
8
```

3. 使用相同的方法查看L2, L3缓存的组数、组相联数和缓存行大小

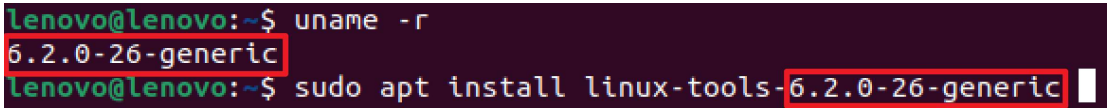
练习4：学习性能分析工具Perf的基本使用

- **任务**：通过 `perf` 观察程序性能。
- **子任务**：
  - 通过 `perf list` 命令查看支持的性能事件；
  - 通过 `perf stat` 查看练习2的矩阵乘法程序的缓存使用情况。
- **目标**：学会 `perf` 的基本使用。

1. 命令安装 `perf`

`perf` 工具包含在 `linux-tools` 中，安装时需保证其版本号与Linux内核版本号一致。

以实验室的Ubuntu双系统为例，安装前执行 `uname -r` 查看内核版本号，再通过 `apt` 命令安装，如下图所示。



实验室Ubuntu双系统的root密码是 `user1234`。

> ⚠ **上图仅为示例，需以命令执行的实际输出为准**
>
> 使用 `apt` 安装 `perf` 时，“`linux-tools-`”后面的版本号信息需以 `uname -r` 的输出为准，请自行作相应的修改。

若提示“`E: Unable to locate package linux-tools-xxx`”，可尝试安装邻近的版本或自行搜索其他解决方案。

2. 查看 `perf` 支持的性能事件

```
sudo perf list
```

若执行失败，可尝试执行 `perf list`。

```
lenovo@10:~$ perf list

List of pre-defined events (to be used in -e):

  duration_time                                      [Tool event]

  branch-instructions OR cpu/branch-instructions/    [Kernel PMU event]
  branch-misses OR cpu/branch-misses/                [Kernel PMU event]
  bus-cycles OR cpu/bus-cycles/                      [Kernel PMU event]
  cache-misses OR cpu/cache-misses/                  [Kernel PMU event]
  cache-references OR cpu/cache-references/          [Kernel PMU event]
  cpu-cycles OR cpu/cpu-cycles/                      [Kernel PMU event]
  cstate_core/c3-residency/                          [Kernel PMU event]
  cstate_core/c6-residency/                          [Kernel PMU event]
  cstate_core/c7-residency/                          [Kernel PMU event]
  cstate_pkg/c10-residency/                          [Kernel PMU event]
  cstate_pkg/c2-residency/                           [Kernel PMU event]
  cstate_pkg/c3-residency/                           [Kernel PMU event]
  cstate_pkg/c6-residency/                           [Kernel PMU event]
  cstate_pkg/c7-residency/                           [Kernel PMU event]
  cstate_pkg/c8-residency/                           [Kernel PMU event]
  cstate_pkg/c9-residency/                           [Kernel PMU event]
  cycles-ct OR cpu/cycles-ct/                        [Kernel PMU event]
  cycles-t OR cpu/cycles-t/                          [Kernel PMU event]
  el-abort OR cpu/el-abort/                          [Kernel PMU event]
  el-capacity OR cpu/el-capacity/                    [Kernel PMU event]
  el-commit OR cpu/el-commit/                        [Kernel PMU event]
  el-conflict OR cpu/el-conflict/                    [Kernel PMU event]
  el-start OR cpu/el-start/                          [Kernel PMU event]
  i915/actual-frequency/                             [Kernel PMU event]
  i915/bcs0-busy/                                    [Kernel PMU event]
  i915/bcs0-sema/                                    [Kernel PMU event]
  i915/bcs0-wait/                                    [Kernel PMU event]
  i915/interrupts/                                   [Kernel PMU event]
  i915/rc6-residency/                                [Kernel PMU event]
  i915/rcs0-busy/                                    [Kernel PMU event]
  i915/rcs0-sema/                                    [Kernel PMU event]
  i915/rcs0-wait/                                    [Kernel PMU event]
  i915/requested-frequency/                          [Kernel PMU event]
  i915/vcs0-busy/                                    [Kernel PMU event]
  i915/vcs0-sema/                                    [Kernel PMU event]
  i915/vcs0-wait/                                    [Kernel PMU event]
  i915/vecs0-busy/                                   [Kernel PMU event]
  i915/vecs0-sema/                                   [Kernel PMU event]
  i915/vecs0-wait/                                   [Kernel PMU event]
  instructions OR cpu/instructions/                  [Kernel PMU event]
  intel_bts//                                        [Kernel PMU event]
  intel_pt//                                         [Kernel PMU event]
```

`perf list` 命令输出的信息较长，每按一下回车就会多输出一行信息。按Q键即可退出。

3. 查看 `lab1_gemm` 程序的缓存使用情况

先通过 `cd` 命令进入 `lab1/build` 文件夹，再执行下列命令查看基本性能事件：

```
sudo perf stat ./dist/bins/lab1_gemm 256 256 256
```

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs/build$ sudo perf stat ./dist/bins/lab1_gemm 256 256 256
[sudo] lenovo 的密码:
GEMM performance info:
                M, K, N: 256, 256, 256
                Ops: 0.0335544
                Total compute time(s): 3.50527
                Cost(s): 0.0175263
                Benchmark(Gflops): 1.91452

 Performance counter stats for './dist/bins/lab1_gemm 256 256 256':

          3,679.90 msec task-clock                #    0.998 CPUs utilized
               280      context-switches          #    0.076 K/sec
                 0      cpu-migrations            #    0.000 K/sec
               313      page-faults               #    0.085 K/sec
    16,377,257,905      cycles                    #    4.450 GHz
    56,534,878,817      instructions              #    3.45  insn per cycle
     3,541,484,913      branches                  #  962.385 M/sec
        13,853,819      branch-misses             #    0.39% of all branches

       3.687359278 seconds time elapsed

       3.680092000 seconds user
       0.000000000 seconds sys
```

添加 `-e` 选项，可查看指定的性能事件。所支持的性能事件可由 `perf list` 查看。

```
sudo perf stat -e L1-dcache-loads,L1-dcache-load-misses,dTLB-loads,dTLB-load-misses ./dist/bins/lab1_gemm 256 256 256
```

```
lenovo@10:~/test1/TA/try/computer_archtecture_labs$ sudo perf stat -e L1-dcache-loads,L1-dcache-load-misses,dTLB-loads,dTLB-load-misses
 ./build/dist/bins/lab1_gemm 256 256 256
GEMM performance info:
               M, K, N: 256, 256, 256
               Ops: 0.0335544
               Total compute time(s): 3.50997
               Cost(s): 0.0175498
               Benchmark(Gflops): 1.91195

 Performance counter stats for './build/dist/bins/lab1_gemm 256 256 256':

     7,065,850,361      L1-dcache-loads
       232,187,035      L1-dcache-load-misses     #    3.29% of all L1-dcache hits
     7,065,850,361      dTLB-loads
             1,249      dTLB-load-misses          #    0.00% of all dTLB cache hits

       3.689693733 seconds time elapsed

       3.689569000 seconds user
       0.000000000 seconds sys
```